

PANDUAN LENGKAP

MEMBANGUN WEBGIS BUSINESS INTELLIGENCE

Dari Nol Hingga Online — Panduan untuk Pemula

ROSYID PAUNDRA

Mencakup: FastAPI · PostgreSQL · PostGIS · Leaflet.js · Supabase · Railway · Vercel · GitHub

Studi Kasus: Kota Yogyakarta

2024 — 2025

Daftar Isi

Bab 1	Pengenalan WebGIS dan Business Intelligence	3
Bab 2	Arsitektur Sistem dan Tools yang Digunakan	5
Bab 3	Persiapan Lingkungan Pengembangan	7
Bab 4	Membangun Database Spasial dengan PostGIS	10
Bab 5	Membangun Backend API dengan FastAPI	13
Bab 6	Membangun Frontend WebGIS dengan Leaflet.js	18
Bab 7	Integrasi ORS API untuk Routing dan Isochrone	22
Bab 8	Deploy ke Internet Gratis	25
Bab 9	Troubleshooting — Masalah Umum dan Solusinya	31
Bab 10	Pengembangan Lebih Lanjut	35

Bab 1 — Pengenalan WebGIS dan Business Intelligence

1.1 Apa itu WebGIS?

WebGIS adalah aplikasi Sistem Informasi Geografis (SIG) yang berjalan di browser web. Berbeda dengan SIG desktop seperti QGIS atau ArcGIS yang harus diinstall, WebGIS bisa diakses oleh siapapun hanya dengan membuka browser dan mengetik URL.

Contoh WebGIS yang sering digunakan sehari-hari adalah Google Maps. Kita bisa melihat peta, mencari rute, dan menemukan tempat — semuanya langsung dari browser tanpa install apapun.

Mengapa Belajar WebGIS?

Visualisasi data lokasi secara interaktif dan mudah dipahami

Analisis spasial untuk pengambilan keputusan bisnis

Dapat diakses siapapun dari mana saja tanpa instalasi

Bisa diintegrasikan dengan data real-time dan database

Permintaan tinggi di industri: pemerintah, startup, logistik, properti

1.2 Apa itu Business Intelligence (BI)?

Business Intelligence adalah proses mengumpulkan, menganalisis, dan menyajikan data bisnis untuk membantu pengambilan keputusan. BI mengubah data mentah menjadi informasi yang bermakna, biasanya dalam bentuk dashboard, grafik, dan laporan.

Ketika BI digabungkan dengan WebGIS, kita mendapat Location-Based BI — kemampuan menganalisis data bisnis berdasarkan lokasi geografis. Contohnya:

- Di mana lokasi terbaik untuk membuka cabang baru?
- Berapa banyak kompetitor dalam radius 1 km dari toko saya?
- Area mana yang belum terlayani oleh bisnis kategori tertentu?
- Seberapa mudah pelanggan menjangkau toko saya?

1.3 Apa yang Akan Kita Bangun?

Dalam panduan ini, kita akan membangun WebGIS BI untuk Kota Yogyakarta yang mencakup:

1	Peta Interaktif Peta gelap modern dengan basemap CartoDB, marker POI berwarna berdasarkan kategori, dan zoom/pan yang smooth.
2	Manajemen POI Sistem untuk menyimpan, menampilkan, dan memfilter titik-titik usaha (Point of Interest) di seluruh kota.
3	Isochrone Analysis Visualisasi area yang bisa dijangkau dalam waktu tertentu dari suatu titik — misalnya area dalam 10 menit berkendara.
4	Site Analysis

	Analisis kelayakan lokasi bisnis baru berdasarkan aksesibilitas jalan, kepadatan kompetitor, dan luas catchment area.
5	Gap Analysis Menemukan area di kota yang kekurangan bisnis kategori tertentu — potensi lokasi baru.
6	Dashboard BI KPI cards, chart distribusi kategori, dan laporan yang bisa di-export sebagai GeoJSON atau CSV.

Bab 2 — Arsitektur Sistem dan Tools yang Digunakan

2.1 Gambaran Arsitektur

Sistem WebGIS BI terdiri dari tiga lapisan utama yang saling berkomunikasi:

Arsitektur 3-Tier

[Browser / Frontend] ↔ [Backend API] ↔ [Database]

Lapisan 1 — Frontend : HTML + CSS + JavaScript + Leaflet.js

Berjalan di browser pengguna

Di-host di Vercel (gratis)

Lapisan 2 — Backend : Python + FastAPI

Memproses request, query DB, panggil ORS API

Di-host di Railway (gratis)

Lapisan 3 — Database : PostgreSQL + PostGIS

Menyimpan data spasial (titik, garis, polygon)

Di-host di Supabase (gratis)

2.2 Penjelasan Setiap Tools

Tools	Kategori	Fungsi dalam Proyek
Python 3.11	Bahasa Pemrograman	Bahasa utama backend — mudah dipelajari, banyak library
FastAPI	Backend Framework	Membuat REST API dengan cepat, dokumentasi otomatis
PostgreSQL	Database	Database relasional yang handal dan gratis
PostGIS	Ekstensi DB	Menambahkan kemampuan spasial ke PostgreSQL (simpan titik, hitung jarak)
Leaflet.js	Library Peta	Library JavaScript untuk membuat peta interaktif di browser
ORS API	Routing Service	Menghitung rute, isochrone, dan matriks jarak — gratis untuk penggunaan terbatas
Supabase	Database Cloud	PostgreSQL + PostGIS gratis di cloud, tidak perlu server sendiri
Railway	Backend Cloud	Menjalankan FastAPI di internet, gratis \$5/bulan
Vercel	Frontend Cloud	Menyajikan file HTML/CSS/JS ke seluruh dunia, gratis
GitHub	Version Control	Menyimpan dan mengelola kode, jembatan antara lokal dan cloud

2.3 Alur Data

Berikut alur ketika pengguna mengklik tombol 'Load POI' di browser:

1	Browser mengirim request JavaScript memanggil: <code>fetch('https://api.railway.app/api/v1/business/poi')</code>
2	FastAPI menerima request Server Python menerima GET <code>/api/v1/business/poi</code> dan memanggil fungsi <code>get_poi()</code>
3	Query ke database FastAPI menjalankan SQL: <code>SELECT id, nama, ST_X(geom) AS lng, ST_Y(geom) AS lat FROM poi</code>
4	Supabase mengembalikan data Database mengirim hasil query berupa array JSON ke FastAPI
5	FastAPI mengirim response FastAPI menformat data dan mengirim JSON kembali ke browser
6	Leaflet menampilkan marker JavaScript membaca JSON dan menempatkan titik-titik di peta Leaflet

Bab 3 — Persiapan Lingkungan Pengembangan

3.1 Software yang Perlu Diinstall

Daftar Software (Semua Gratis)

1. Python 3.11 — python.org/downloads/release/python-3119
2. Git — git-scm.com/downloads
3. PostgreSQL 16 — postgresql.org/download/windows
4. pgAdmin 4 — pgadmin.org (biasanya sudah bundel dengan PostgreSQL)
5. VS Code — code.visualstudio.com (editor kode, opsional tapi sangat disarankan)

3.2 Install Python 3.11

PENTING: Jangan pakai Python 3.13 atau yang lebih baru — banyak library GIS belum mendukungnya. Selalu gunakan Python 3.11.

1. Buka python.org/downloads/release/python-3119
2. Scroll ke bawah, klik 'Windows installer (64-bit)'
3. Jalankan file installer yang terdownload
4. CENTANG 'Add Python to PATH' di halaman pertama — ini penting!
5. Klik 'Customize installation' → centang semua → Next
6. Klik Install → tunggu selesai

Verifikasi instalasi — buka Command Prompt (tekan Windows+R, ketik cmd, Enter):

```
python --version
# Harus muncul: Python 3.11.x
```

3.3 Install Git

7. Download dari git-scm.com/downloads
8. Jalankan installer, klik Next terus dengan pengaturan default
9. Verifikasi: buka Command Prompt baru, ketik:

```
git --version
# Harus muncul: git version 2.x.x
```

10. Konfigurasi identitas Git (wajib untuk commit):

```
git config --global user.name "Nama Kamu"
git config --global user.email "email@kamu.com"
```

3.4 Install PostgreSQL dan PostGIS

11. Download PostgreSQL dari postgresql.org/download/windows
12. Jalankan installer — catat password yang kamu set untuk user 'postgres'
13. Saat ditanya komponen, pastikan centang: PostgreSQL Server, pgAdmin 4, Stack Builder
14. Setelah install selesai, Stack Builder akan terbuka — pilih 'Spatial Extensions' → PostGIS
15. Jika Stack Builder tidak otomatis buka, buka pgAdmin → klik kanan database → Query Tool → jalankan:

```
CREATE EXTENSION IF NOT EXISTS postgis;  
SELECT PostGIS_Version();
```

Tips: Ingat Password PostgreSQL

Password yang kamu set saat install PostgreSQL akan dibutuhkan berkali-kali. Simpan di tempat yang mudah diingat. Jika lupa, bisa direset lewat pgAdmin.

3.5 Setup Struktur Folder Proyek

Buat struktur folder berikut di komputer:

```
C:\Users\NamaKamu\webgis-bi\  
    backend\  
        main.py  
        requirements.txt  
        .env  
        Procfile  
        runtime.txt  
    frontend\  
        index.html  
    .gitignore  
    README.md
```

Buat folder lewat Command Prompt:

```
mkdir C:\Users\%USERNAME%\webgis-bi  
mkdir C:\Users\%USERNAME%\webgis-bi\backend  
mkdir C:\Users\%USERNAME%\webgis-bi\frontend  
cd C:\Users\%USERNAME%\webgis-bi
```

3.6 Setup Virtual Environment Python

Virtual environment adalah 'kotak pasir' Python yang terisolasi untuk setiap proyek. Ini mencegah konflik antar library.

```
cd C:\Users\NamaKamu\webgis-bi  
  
# Buat virtual environment  
python -m venv venv  
  
# Aktifkan (Windows)  
venv\Scripts\activate  
  
# Prompt berubah jadi: (venv) C:\...>  
# Ini tanda virtual environment sudah aktif
```

Penting: Selalu Aktifkan venv

Setiap kali membuka terminal baru untuk bekerja di proyek ini, selalu jalankan: `venv\Scripts\activate`
Kalau tidak, library yang diinstall tidak akan ditemukan.

Bab 4 — Membangun Database Spasial dengan PostGIS

4.1 Konsep Database Spasial

Database biasa menyimpan angka, teks, dan tanggal. Database spasial (spatial database) bisa menyimpan geometri — titik (point), garis (line), dan polygon. PostGIS adalah ekstensi yang menambahkan kemampuan ini ke PostgreSQL.

Contoh perbandingan:

Data	Database Biasa	Database Spasial
Lokasi toko	Simpan sebagai teks: 'Jl. Malioboro 12'	Simpan sebagai titik: POINT(110.3644 -7.7925)
Jarak antar toko	Tidak bisa dihitung langsung	ST_Distance(geom_a, geom_b)
Toko dalam radius	Tidak bisa langsung	ST_DWithin(geom, titik, radius)

4.2 Membuat Database

Buka pgAdmin → klik kanan 'Databases' → Create → Database. Beri nama 'webgis'. Atau lewat Query Tool:

```
-- Buka pgAdmin → Query Tool → jalankan:  
CREATE DATABASE webgis;
```

Aktifkan PostGIS di database baru:

```
-- Sambungkan ke database webgis dulu  
\c webgis  
  
-- Aktifkan ekstensi PostGIS  
CREATE EXTENSION IF NOT EXISTS postgis;  
  
-- Verifikasi  
SELECT PostGIS_Version();  
-- Harus muncul: '3.x USE_GEOS=1 ...'
```

4.3 Membuat Tabel poi (Point of Interest)

Tabel poi menyimpan semua titik usaha/lokasi bisnis di kota. Kolom geom menyimpan koordinat sebagai tipe data spasial.

```
CREATE TABLE poi (  
  id          SERIAL PRIMARY KEY,  
  nama        TEXT NOT NULL,      -- Nama usaha  
  kategori    TEXT,              -- kuliner, retail, kesehatan, dst  
  alamat      TEXT,              -- Alamat lengkap  
  kecamatan   TEXT,              -- Nama kecamatan  
  metadata    JSONB DEFAULT '{}', -- Data tambahan bebas  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  geom        GEOMETRY(Point, 4326) -- Koordinat WGS84  
);
```

```
-- Index spasial untuk query yang cepat
CREATE INDEX ON poi USING GIST(geom);
```

4.4 Membuat Tabel roads (Jaringan Jalan)

```
CREATE TABLE roads (
  id          SERIAL PRIMARY KEY,
  name        TEXT,
  type        TEXT,      -- primary, secondary, residential, dst
  oneway      BOOLEAN DEFAULT false,
  maxspeed    INTEGER,
  geom        GEOMETRY(LineString, 4326)
);

CREATE INDEX ON roads USING GIST(geom);
```

4.5 Insert Data Contoh

Gunakan fungsi ST_SetSRID dan ST_MakePoint untuk memasukkan data koordinat.
Format: ST_MakePoint(LONGITUDE, LATITUDE) — perhatikan urutan lng dulu, baru lat.

```
INSERT INTO poi (nama, kategori, alamat, kecamatan, geom) VALUES
('Warung Bu Tini', 'kuliner', 'Jl. Malioboro 12', 'Gedongtengen',
 ST_SetSRID(ST_MakePoint(110.3644, -7.7925), 4326)),

('Toko Batik Sari', 'retail', 'Jl. Prawirodaman 8', 'Mergangsan',
 ST_SetSRID(ST_MakePoint(110.3701, -7.8054), 4326)),

('Klinik Sehat', 'kesehatan', 'Jl. Veteran 5', 'Gondokusuman',
 ST_SetSRID(ST_MakePoint(110.3765, -7.7845), 4326));
```

4.6 Query Spasial Dasar

Berikut beberapa query PostGIS yang sering digunakan:

```
-- 1. Ambil semua POI dengan koordinat
SELECT nama, kategori, ST_X(geom) AS lng, ST_Y(geom) AS lat
FROM poi;

-- 2. POI dalam radius 1 km dari titik tertentu
SELECT nama, ST_Distance(geom::geography,
  ST_SetSRID(ST_MakePoint(110.3644, -7.7925), 4326)::geography) AS jarak_meter
FROM poi
WHERE ST_DWithin(geom::geography,
  ST_SetSRID(ST_MakePoint(110.3644, -7.7925), 4326)::geography, 1000)
ORDER BY jarak_meter;

-- 3. Hitung total POI per kategori
SELECT kategori, COUNT(*) AS total
FROM poi GROUP BY kategori ORDER BY total DESC;
```

Bab 5 — Membangun Backend API dengan FastAPI

5.1 Apa itu REST API?

API (Application Programming Interface) adalah jembatan komunikasi antar program. REST API adalah API yang berkomunikasi lewat HTTP — protokol yang sama digunakan browser untuk mengakses website.

Cara kerjanya sederhana: Frontend mengirim request ke URL tertentu, Backend memproses dan mengembalikan data JSON.

Contoh Request dan Response

Request : GET <https://api.kamu.com/api/v1/business/poi?kategori=kuliner>

Response :

```
[
  { "id": 1, "nama": "Warung Bu Tini", "lat": -7.7925, "lng": 110.3644 },
  { "id": 2, "nama": "RM Padang", "lat": -7.7812, "lng": 110.3792 }
]
```

5.2 Install Dependencies

Buat file requirements.txt di folder backend/ dengan isi:

```
fastapi==0.111.0
uvicorn[standard]==0.30.1
httpx==0.27.0
psycpg2-binary==2.9.9
pydantic==2.7.1
python-dotenv==1.0.1
```

Install semua dependencies:

```
cd backend
venv\Scripts\activate
pip install -r requirements.txt
```

5.3 Konfigurasi dengan File .env

File .env menyimpan konfigurasi sensitif seperti password database. File ini TIDAK boleh diupload ke GitHub.

Buat file .env di folder backend/:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=webgis
DB_USER=postgres
DB_PASS=password_postgres_kamu
ORS_BASE_URL=https://api.openrouteservice.org
ORS_API_KEY=api_key_ors_kamu
```

5.4 Struktur main.py

Berikut kerangka utama file main.py:

```

from fastapi import FastAPI, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import psycopg2, psycopg2.extras, httpx, os, json
from dotenv import load_dotenv
from pathlib import Path

# Load konfigurasi dari .env
load_dotenv(Path(__file__).parent / '.env')

# Baca variabel environment
DB_HOST = os.getenv('DB_HOST', 'localhost')
DB_PORT = os.getenv('DB_PORT', '5432')
DB_NAME = os.getenv('DB_NAME', 'webgis')
DB_USER = os.getenv('DB_USER', 'postgres')
DB_PASS = os.getenv('DB_PASS')
ORS_BASE_URL = os.getenv('ORS_BASE_URL')
ORS_API_KEY = os.getenv('ORS_API_KEY')

# Buat aplikasi FastAPI
app = FastAPI(title='WebGIS BI Jogja', version='1.0.0')

# Izinkan akses dari frontend (CORS)
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
    allow_methods=['*'],
    allow_headers=['*'],
)

# Fungsi koneksi database
def get_db():
    return psycopg2.connect(
        host=DB_HOST, port=DB_PORT, dbname=DB_NAME,
        user=DB_USER, password=DB_PASS,
        connect_timeout=5, sslmode='prefer'
    )

```

5.5 Contoh Endpoint GET /business/poi

```

@app.get('/api/v1/business/poi')
async def get_poi(
    kategori: str = None, # Parameter opsional
    limit: int = 200
):
    conn = get_db()
    cur = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)

    if kategori:
        cur.execute(
            'SELECT id, nama, kategori, alamat,
            ST_X(geom) AS lng, ST_Y(geom) AS lat

```

```

        FROM poi WHERE kategori = %s LIMIT %s',
        (kategori, limit)
    )
else:
    cur.execute(
        'SELECT id, nama, kategori, alamat,
        ST_X(geom) AS lng, ST_Y(geom) AS lat
        FROM poi LIMIT %s',
        (limit,)
    )

rows = cur.fetchall()
cur.close()
conn.close()
return rows # FastAPI otomatis konversi ke JSON

```

5.6 Menjalankan Backend Lokal

```

cd backend
venv\Scripts\activate
uvicorn main:app --reload --port 8001

# Output yang diharapkan:
# INFO: Uvicorn running on http://127.0.0.1:8001
# INFO: Application startup complete.

```

Buka browser, akses <http://localhost:8001/docs> — akan muncul halaman Swagger UI dengan semua endpoint yang terdaftar dan bisa dicoba langsung.

Bab 6 — Membangun Frontend WebGIS dengan Leaflet.js

6.1 Struktur Dasar index.html

Seluruh frontend WebGIS ada dalam satu file HTML. Tidak perlu build tools, tidak perlu Node.js — cukup satu file yang bisa dibuka di browser.

```
<!DOCTYPE html>
<html lang='id'>
<head>
  <meta charset='UTF-8' />
  <meta name='viewport' content='width=device-width, initial-scale=1.0' />
  <title>WebGIS BI Jogja</title>

  <!-- 1. Load Leaflet CSS (WAJIB sebelum JS) -->
  <link rel='stylesheet'
        href='https://unpkg.com/leaflet@1.9.4/dist/leaflet.css' />

  <!-- 2. Load Leaflet JS -->
  <script src='https://unpkg.com/leaflet@1.9.4/dist/leaflet.js'></script>

  <style>
    /* WAJIB: Peta tidak muncul tanpa height! */
    html, body { height: 100%; margin: 0; }
    #map { width: 100%; height: 100vh; }
  </style>
</head>
<body>
  <div id='map'></div>
  <script>
    // Inisialisasi peta
    const map = L.map('map').setView([-7.7956, 110.3653], 13);

    // Tambah basemap
    L.tileLayer(
      'https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png',
      { maxZoom: 19 }
    ).addTo(map);
  </script>
</body>
</html>
```

6.2 Menampilkan Marker POI dari API

```
const API = 'http://localhost:8001/api/v1';

async function loadPOI() {
  // Ambil data dari backend
  const response = await fetch(API + '/business/poi');
```

```

const data = await response.json();

// Warna berdasarkan kategori
const warna = {
  kuliner: '#00e5a0',
  retail: '#0080ff',
  kesehatan: '#ff4d4d',
  pendidikan: '#a78bfa',
};

// Tambah marker ke peta
data.forEach(poi => {
  const c = warna[poi.kategori] || '#ffffff';

  // Custom icon berupa lingkaran berwarna
  const icon = L.divIcon({
    html: `<div style='width:10px;height:10px;
      border-radius:50%;background:${c}'></div>`,
    className: '',
    iconSize: [10, 10]
  });

  L.marker([poi.lat, poi.lng], { icon })
    .bindPopup(`<b>${poi.nama}</b><br>${poi.kategori}`)
    .addTo(map);
});
}

// Panggil saat halaman dimuat
window.addEventListener('load', loadPOI);

```

Kesalahan Umum: Peta Tidak Muncul

Masalah 1: Div peta tidak punya height → Tambahkan: `#map { height: 100vh; }`

Masalah 2: Dibuka via `file://` bukan `http://` → Jalankan: `python -m http.server 3000`

Masalah 3: Leaflet CSS tidak dimuat → Pastikan `<link>` CSS ada sebelum `<script>` JS

Cara debug: Buka F12 → Console → jalankan:
`document.getElementById('map').offsetHeight`

Jika hasilnya 0, berarti div tidak punya tinggi.

6.3 Menjalankan Frontend Lokal

Jangan buka file HTML dengan klik dua kali — harus lewat HTTP server agar semua fitur bekerja.

```

cd frontend
python -m http.server 3000

# Buka browser: http://localhost:3000
# BUKAN: file:///C:/Users/.../index.html

```

Bab 7 — Integrasi ORS API untuk Routing dan Isochrone

7.1 Apa itu OpenRouteService (ORS)?

OpenRouteService adalah layanan routing gratis berbasis data OpenStreetMap. Layanan ini menyediakan tiga fitur utama yang kita butuhkan:

Fitur	Endpoint	Fungsi
Routing	/v2/directions/{profile}	Hitung rute terpendek/tercepat antara dua titik
Isochrone	/v2/isochrones/{profile}	Hitung area yang bisa dijangkau dalam waktu tertentu
Matrix	/v2/matrix/{profile}	Hitung jarak/waktu antara N titik sekaligus

7.2 Mendapatkan API Key ORS

1	Buka openrouteservice.org Klik tombol 'Sign Up' di pojok kanan atas halaman
2	Daftar dengan email Isi form registrasi, verifikasi email yang dikirim
3	Login dan buka Dashboard Setelah login, klik 'Dashboard' di menu atas
4	Request a Token Klik 'Request a token', beri nama token (misal: webgis-jogja), klik Create
5	Copy API Key API key akan muncul — copy dan simpan di file .env sebagai ORS_API_KEY

7.3 Implementasi Endpoint Isochrone di FastAPI

```
@app.get('/api/v1/geo/isochrone')
async def get_isochrone(
    lng: float,          # Longitude titik pusat
    lat: float,          # Latitude titik pusat
    range_minit: int = 10, # Jangkauan dalam menit
    profile: str = 'driving-car' # Mode transport
):
    # Siapkan payload untuk ORS
    payload = {
        'locations': [[lng, lat]],
        'range': [range_minit * 60], # ORS pakai detik
        'range_type': 'time',
    }

    # Kirim request ke ORS Cloud
    async with httpx.AsyncClient(timeout=30) as client:
        response = await client.post(
```

```

        f'{ORS_BASE_URL}/v2/isochrones/{profile}',
        headers={
            'Content-Type': 'application/json',
            'Accept': 'application/geo+json', # WAJIB untuk ORS
            'Authorization': ORS_API_KEY
        },
        json=payload
    )

    if response.status_code != 200:
        raise HTTPException(response.status_code, f'ORS error: {response.text}')

    return response.json()

```

7.4 Menampilkan Isochrone di Leaflet

```

// Variabel global untuk menyimpan layer isochrone
let isoLayer = null;

map.on('click', async function(e) {
    const { lat, lng } = e.latlng;

    // Hapus isochrone sebelumnya
    if (isoLayer) map.removeLayer(isoLayer);

    // Ambil isochrone dari backend
    const res = await fetch(
        `${API}/geo/isochrone?lng=${lng}&lat=${lat}&range_menit=10`
    );
    const data = await res.json();

    // Tampilkan polygon di peta
    isoLayer = L.geoJSON(data, {
        style: {
            color: '#0080ff',
            fillColor: '#0080ff',
            fillOpacity: 0.15,
            weight: 2
        }
    }).addTo(map);

    // Zoom ke area isochrone
    map.fitBounds(isoLayer.getBounds());
});

```

Bab 8 — Deploy ke Internet Gratis

8.1 Overview Strategi Deploy

Arsitektur Deploy (Semua Gratis)

Kode → GitHub (penyimpanan dan version control)

Database → Supabase (PostgreSQL + PostGIS cloud)

Backend → Railway (menjalankan FastAPI Python)

Frontend → Vercel (menyajikan HTML/CSS/JS)

Alur kerja: Edit kode lokal → git push → Railway + Vercel otomatis rebuild

8.2 Push Kode ke GitHub

1	Buat akun GitHub Daftar di github.com jika belum punya akun
2	Buat repository baru Klik '+' → New repository → nama: webgis-bi → Public → Create (jangan centang Initialize)
3	Buat .gitignore Buat file .gitignore di root proyek dengan isi: venv/, __pycache__/, .env, *.pyc
4	Init Git dan commit pertama Jalankan: git init → git add backend/ frontend/ .gitignore README.md → git commit -m 'initial commit'
5	Hubungkan ke GitHub dan push git remote add origin https://github.com/USERNAME/webgis-bi.git → git branch -M main → git push -u origin main

PENTING: Jangan Upload .env ke GitHub

File .env berisi password database dan API key.

Pastikan .env ada di .gitignore sebelum push!

Periksa: jalankan 'git status' — .env tidak boleh muncul dalam daftar.

8.3 Setup Database di Supabase

1	Daftar di supabase.com Klik Sign Up → Continue with GitHub → gratis, tidak perlu kartu kredit
2	Buat project baru New Project → isi nama → set password (CATAT!) → Region: Southeast Asia → Create
3	Aktifkan PostGIS SQL Editor → New Query → jalankan: CREATE EXTENSION IF NOT EXISTS postgis;
4	Buat tabel SQL Editor → paste SQL CREATE TABLE roads dan poi → Run

5

Catat credentials

Settings → Database → Connection Pooling → catat Host, Port (6543), Database, Username, Password

Gunakan Transaction Pooler, Bukan Direct Connection

Supabase punya dua mode koneksi:

- Direct Connection (port 5432): Hanya IPv6 — TIDAK bisa dari Railway
- Transaction Pooler (port 6543): IPv4 dan IPv6 — BISA dari Railway

Selalu gunakan port 6543 dan host dari bagian 'Session Pooler' atau 'Transaction Pooler' di Settings Database Supabase.

DB_HOST = aws-0-ap-southeast-1.pooler.supabase.com

DB_PORT = 6543

DB_USER = postgres.PROJECT_ID (bukan hanya 'postgres!')

8.4 Deploy Backend ke Railway

Railway menjalankan FastAPI Python di cloud. Gratis \$5 credit/bulan — cukup untuk project kecil.

1

Daftar di railway.app

Sign up with GitHub — tidak perlu kartu kredit

2

New Project → Deploy from GitHub

Pilih repo webgis-bi → Railway akan detect otomatis

3

Konfigurasi service

Settings → Root Directory: backend → Start Command: uvicorn main:app --host 0.0.0.0 --port \$PORT

4

Tambah Environment Variables

Tab Variables → tambahkan DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASS, ORS_BASE_URL, ORS_API_KEY

5

Generate domain publik

Settings → Domains → Generate Domain → dapat URL seperti: nama-project.up.railway.app

6

Verifikasi deploy berhasil

Buka <https://nama-project.up.railway.app/docs> → harus muncul halaman Swagger

8.5 Deploy Frontend ke Vercel

1

Update URL API di index.html

Ganti const API = 'http://localhost:8001/api/v1' menjadi URL Railway kamu

2

Push perubahan ke GitHub

git add frontend/index.html → git commit -m 'update API URL' → git push

3

Daftar di vercel.com

Sign up with GitHub — gratis, tidak perlu kartu kredit

4

Import project

5

Add New Project → Import repo webgis-bi → Root Directory: frontend → Deploy

Test URL Vercel

Buka URL yang diberikan Vercel → peta harus muncul dan data POI terbaca

8.6 Alur Update Kode Setelah Deploy

Setelah sistem live, setiap perubahan kode cukup:

```
# 1. Edit file yang diperlukan
# 2. Simpan perubahan ke Git
git add .
git commit -m 'deskripsi perubahan kamu'
git push

# Railway otomatis rebuild backend (~2 menit)
# Vercel otomatis rebuild frontend (~1 menit)
```

Bab 9 — Troubleshooting: Masalah Umum dan Solusinya

Pesan Error	Penyebab	Solusi
python is not recognized	Python tidak ada di PATH	Install ulang Python, centang 'Add to PATH'
Failed building wheel for pydantic-core	Python terlalu baru (3.13+)	Gunakan Python 3.11 — download versi ini khusus
Map container is already initialized	L.map() dipanggil dua kali	Simpan instance map di variabel, cek sebelum init
No 'Access-Control-Allow-Origin'	CORS belum dikonfigurasi di backend	Tambahkan CORSMiddleware ke FastAPI app
ERR_CONNECTION_REFUSED port 8001	Backend belum jalan	Jalankan: uvicorn main:app --reload --port 8001
404 Not Found di Swagger	URL endpoint salah	Cek prefix /api/v1/ sudah benar di index.html
psycopg2 connection refused	PostgreSQL tidak jalan	Jalankan: net start postgresql-x64-16
function round(double precision) does not exist	Cast tipe data salah	Tambahkan CAST(...AS numeric) sebelum ROUND()
Network is unreachable ke Supabase	Railway pakai IPv4, Supabase Direct pakai IPv6	Gunakan Transaction Pooler Supabase (port 6543)
could not translate host name	Spasi di awal hostname di Railway Variables	Hapus spasi di depan nilai DB_HOST
Peta tidak muncul di browser	Div peta tingginya 0px	Tambahkan: #map { height: 100vh; }
ORS error 406 Not Acceptable	Header Accept salah	Ganti Accept: 'application/geo+json' (bukan application/json)
git push rejected (fetch first)	Remote ada perubahan yang belum ditarik	Jalankan: git pull origin main --rebase → git push
backend adalah nested git repo	Folder backend punya .git sendiri	Jalankan: rmdir /s /q backend\.git → git add backend/

9.2 Cara Debug yang Efektif

Urutan yang benar saat ada error:

16. Buka F12 → Console di browser — baca pesan error pertama dengan teliti
17. Buka terminal backend (uvicorn) — lihat traceback merah setelah request
18. Buka Swagger UI (/docs) — coba execute endpoint yang bermasalah
19. Isolasi masalah: apakah dari frontend, backend, atau database?
20. Cari pesan error spesifik di Google — biasanya ada solusi di Stack Overflow

Perintah Debug yang Berguna

```
# Cek Python version
```

```
python --version
```

```
# Cek koneksi database
```

```
python -c "import psycopg2; psycopg2.connect(host='localhost', dbname='webgis',  
user='postgres', password='pass'); print('OK')"
```

```
# Cek tinggi div peta (di Console browser)
```

```
document.getElementById('map').offsetHeight
```

```
# Cek backend sudah jalan
```

```
curl http://localhost:8001/api/v1/report/dashboard
```

Bab 10 — Pengembangan Lebih Lanjut

10.1 Menambahkan Data Jalan dari OpenStreetMap

Data jalan Kota Yogyakarta bisa didapat gratis dari OpenStreetMap menggunakan tool `osm2pgsql` atau QGIS.

1	Download data OSM Buka <code>download.geofabrik.de</code> → Indonesia → download file <code>.pbf</code> untuk Jawa Tengah/DIY
2	Install QGIS Download di <code>qgis.org</code> — gratis. QGIS bisa membaca file OSM dan export ke PostgreSQL
3	Import ke PostgreSQL QGIS → Layer → Add Layer → Vector → pilih file <code>.pbf</code> → pilih layer 'lines' → Export ke database webgis, tabel roads
4	Filter jalan saja SQL: <code>DELETE FROM roads WHERE highway IS NULL;</code> — hapus fitur non-jalan

10.2 Menambahkan Autentikasi

Untuk membatasi akses ke API, tambahkan JWT authentication:

```
pip install python-jose[cryptography] passlib[bcrypt]

# Di main.py tambahkan endpoint login:
@app.post('/token')
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    # Verifikasi username dan password
    # Return JWT token
    ...
```

10.3 Optimasi Performa

- Tambahkan caching dengan Redis untuk query yang sering diulang
- Gunakan connection pooling (`psycopg2.pool`) untuk database
- Pagination pada endpoint yang mengembalikan banyak data
- Kompresi gzip pada response API
- Lazy loading marker — hanya tampilkan POI yang ada di viewport peta

10.4 Fitur Lanjutan yang Bisa Ditambahkan

Fitur	Deskripsi
Upload Data	Form untuk upload file CSV/GeoJSON dan import massal ke database
Pencarian Lokasi	Integrasi Nominatim API (geocoding gratis) untuk search by alamat
Filter Tanggal	Filter POI berdasarkan tanggal dibuat — berguna untuk analisis tren
Perbandingan Area	Side-by-side comparison dua atau lebih kecamatan

Fitur	Deskripsi
Export PDF	Generate laporan PDF dari data yang ditampilkan di peta
Notifikasi Real-time	WebSocket untuk update data secara live tanpa refresh
Mobile App	Wrap WebGIS ke dalam PWA (Progressive Web App) agar bisa diinstall di HP

10.5 Referensi dan Sumber Belajar

- Dokumentasi FastAPI: fastapi.tiangolo.com
- Dokumentasi Leaflet: leafletjs.com/reference.html
- Tutorial PostGIS: postgis.net/workshops
- OpenRouteService API: openrouteservice.org/dev/#/api-docs
- Supabase Docs: supabase.com/docs
- Railway Docs: docs.railway.app
- Vercel Docs: vercel.com/docs

Selamat! Kamu telah berhasil membangun WebGIS BI dari nol.

Sistem yang kamu bangun mencakup backend Python, database spasial, peta interaktif, analisis lokasi berbasis ORS, dan deployment ke cloud — semua gratis.